MoodLift

Software Requirements Specification

A Modern Frontend-Based Activity Suggestion & Recommendation Platform

Document Type	Software Requirements Specification (SRS)
Submitted By	Nikhil Kumar
Roll No.	22515000088
Branch	Computer Science & Engineering (Core)
Institution	GLA University, Greater Noida
Version	1.0
Date	18 August 2026
Standard Followed	IEEE 830 / IEEE 29148 (adapted)

Student Declaration

I, **Nikhil Kumar**, Roll No. **22515000088**, a student of B.Tech — Computer Science & Engineering (Core) at **GLA University, Greater Noida**, hereby declare that this Software Requirements Specification document, prepared for the project titled **"MoodLift — Activity Suggestion & Recommendation Platform,"** is an original work carried out by me as part of the software engineering project requirement. This document has been prepared under the guidance of my faculty mentor and represents my own understanding and analysis of the system requirements.

Student Name	Nikhil Kumar
Roll Number	22515000088
Branch	Computer Science & Engineering (Core)
Institution	GLA University, Greater Noida
Project Title	MoodLift — Activity Suggestion Platform
Date	18 August 2026

_____ _____

Signature of Student **Signature of Faculty Mentor**

Document Control

Revision History

Version	Date	Author	Description
0.1	01 Aug 2026	Nikhil Kumar	Initial draft — scope and feature list
0.5	10 Aug 2026	Nikhil Kumar	Added functional requirements and use cases
1.0	18 Aug 2026	Nikhil Kumar	Finalized SRS for faculty submission

Approval

Role	Name	Signature	Date
Project Guide / Faculty Mentor			
Student Developer	Nikhil Kumar (22515000088)		

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of **MoodLift**, a frontend-based activity suggestion and recommendation platform. The purpose of this document is to provide a clear, complete, and unambiguous understanding of the system's intended behaviour so that it can be used as a reference by students during design and development, by faculty during evaluation, and by any future contributor extending the project. This document follows the general structure recommended by IEEE 830 / IEEE 29148, adapted to a scope appropriate for an academic project.

1.2 Scope

MoodLift is a web-based application that helps a user quickly decide "what to do right now" by suggesting simple activities (for example: a short walk, journaling, a breathing exercise, listening to music, a quick workout, reading, or a creative task) based on the user's current mood, available time, and preferences. The initial version of the system is intentionally kept **frontend-focused**: the recommendation logic runs on the client side using a curated, rule-based activity dataset bundled with the application, and user data (favourites, history, mood log) is stored locally in the browser. The system is designed so that a backend/API and database layer (described in Section 6 and Section 7) can be integrated in a later phase without changing the core user experience.

In scope for the current phase:

- Capturing the user's mood/energy level and available time through a simple guided interface.
- Generating activity suggestions using a client-side recommendation/filtering algorithm.
- Allowing users to browse, search, and filter the full activity catalogue.
- Letting users mark activities as favourites and view a short history of completed suggestions.
- Collecting lightweight feedback (thumbs up/down or star rating) to refine future suggestions.
- A responsive, accessible, and visually calming user interface suitable for daily use.

Out of scope for the current phase (identified as future work in Section 11):

- Multi-user accounts with server-side authentication and cloud data sync.
- Machine-learning based personalization trained on large user datasets.
- Integration with third-party calendar, fitness, or social media platforms.
- Native mobile applications (Android/iOS) — the current scope targets a responsive web app only.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Activity	A single suggested task or exercise (e.g., "10-minute walk", "5-minute breathing exercise") stored in the activity catalogue.
Recommendation Engine	The client-side logic module that filters and ranks activities based on user mood, time, and preference inputs.
Mood Input	The user-selected emotional/energy state (e.g., Stressed, Low Energy, Happy, Bored) used as a primary filter for recommendations.
LocalStorage	Browser-based key-value storage used to persist user preferences, favourites, and history on the client device.
UI / UX	User Interface / User Experience
API	Application Programming Interface — planned for the future backend integration phase.
Responsive Design	A UI design approach that adapts layout to different screen sizes (mobile, tablet, desktop).

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Requirements Engineering.
- Nielsen Norman Group — Usability Heuristics for User Interface Design.
- Project team meeting notes and internal brainstorming documents (MoodLift, 2026).

1.5 Document Overview

Section 2 describes the overall context of the product. Section 3 lists the detailed functional requirements organised as system features. Section 4 summarizes the use case model. Section 5 covers external interface requirements. Section 6 presents the proposed system architecture. Section 7 defines the data requirements for both the current local-storage model and the future database. Section 8 specifies non-functional requirements. Sections 9–12 cover other requirements, testing considerations, future scope, and supporting appendices.

2. Overall Description

2.1 Product Perspective

MoodLift is a new, self-contained frontend application. It is not a modification of an existing product and has no mandatory dependency on an external backend in its current phase. The system is structured internally into three logical layers so that it can evolve smoothly:

Layer	Responsibility	Current Phase
Presentation Layer	React-based UI components: mood selector, suggestion cards, catalogue, favourites, settings.	Implemented
Logic Layer	Recommendation engine, filtering rules, state management.	Implemented (client-side)
Data Layer	Activity catalogue (static JSON) + browser LocalStorage for user data.	Implemented; DB/API planned

2.2 Product Functions

A high-level summary of the major functions provided by MoodLift:

- **Guided Mood Check-in:** Ask the user a few quick questions (current mood, energy level, available time).
- **Smart Suggestion:** Instantly suggest one or more matching activities from the catalogue.
- **Browse Catalogue:** Let users explore all available activities with filters (category, duration, mood tag).
- **Favourites & History:** Save preferred activities and view previously suggested/completed activities.
- **Feedback Loop:** Collect simple feedback to bias future suggestions toward liked activities.
- **Personal Settings:** Let users set preferences such as favourite categories or activities to avoid.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
General User	Any individual looking for a quick activity suggestion to improve mood, productivity, or break a boredom/stress cycle.	Low — basic web/app familiarity
Student User	Students using the app during study breaks to manage stress and stay productive.	Low to Moderate
Administrator (Content Maintainer)	Project team member who curates and updates the activity catalogue (JSON dataset) during development.	Moderate — comfortable editing structured data/code

2.4 Operating Environment

- **Client:** Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, tablet, or mobile, with JavaScript enabled.
- **Development Stack:** HTML5, CSS3, JavaScript (ES6+), and a modern frontend framework (React.js recommended).

- **Hosting (current phase):** Static hosting is sufficient (e.g., Netlify, Vercel, GitHub Pages) since no server-side processing is required.
- **Future Phase:** A lightweight backend (e.g., Node.js/Express) with a database (e.g., MongoDB or PostgreSQL) for multi-device sync.

2.5 Design and Implementation Constraints

- The project must be completed within a single academic semester by a small student team (typically 2–5 members).
- The current phase must not require a paid backend/database subscription; all data persistence uses browser LocalStorage.
- The UI must be responsive and usable on both desktop and mobile screens without separate codebases.
- The recommendation logic must run entirely on the client to avoid infrastructure and hosting costs during evaluation.
- The system should be built using open-source, freely available tools and libraries only.

2.6 Assumptions and Dependencies

- Users have access to a device with a modern browser and a stable internet connection to load the app.
- The activity catalogue is curated and reviewed by the project team; it does not rely on external live data sources.
- It is assumed that a single browser/device is used per user in the current phase (no cross-device sync).
- Faculty evaluators will access the application via a shared link or local demo during presentation.

3. System Features (Functional Requirements)

This section lists the functional requirements grouped by feature. Each requirement is assigned a unique ID (FR-x.y) and a priority: **High** (must-have for MVP), **Medium** (should-have), or **Low** (nice-to-have / stretch goal).

3.1 Mood / Preference Input

Allows the user to describe how they feel and how much time they have, forming the basis for recommendations.

ID	Requirement Description	Priority
FR-1.1	The system shall display a mood selection screen with a set of predefined mood options (e.g., Stressed, Sad, Happy, Bored, Tired, Anxious, Energetic).	High
FR-1.2	The system shall allow the user to select an available time range (e.g., 5 min, 15 min, 30 min, 1 hour+).	High
FR-1.3	The system shall allow the user to optionally select a preferred activity category (e.g., Physical, Mental, Creative, Social, Relaxation).	Medium
FR-1.4	The system shall validate that at least a mood is selected before generating a suggestion.	High

3.2 Activity Recommendation Engine

The core logic that matches user input to suitable activities from the catalogue.

ID	Requirement Description	Priority
FR-2.1	The system shall filter the activity catalogue based on the selected mood tag(s), time availability, and category.	High
FR-2.2	The system shall rank filtered results using a weighted scoring rule (mood match, past feedback, favourite status).	High
FR-2.3	The system shall present the top suggestion prominently, with an option to view alternative suggestions ("Show another").	High
FR-2.4	The system shall avoid repeating the same activity suggested in the immediately preceding session, where alternatives exist.	Medium
FR-2.5	The system shall complete the recommendation calculation and display results within 1 second of user input (client-side only).	High

3.3 Activity Catalogue Browsing

ID	Requirement Description	Priority
FR-3.1	The system shall provide a catalogue view listing all available activities.	High
FR-3.2	The system shall allow the user to search activities by keyword/title.	Medium
FR-3.3	The system shall allow the user to filter the catalogue by category, mood tag, and duration.	Medium
FR-3.4	The system shall display, for each activity, a title, short description, category, estimated duration, and an icon/illustration.	High

3.4 Favourites and History

ID	Requirement Description	Priority
FR-4.1	The system shall allow the user to mark/unmark an activity as a favourite.	High
FR-4.2	The system shall display a dedicated "Favourites" screen listing all favourited activities.	High
FR-4.3	The system shall maintain a chronological history of activities that were suggested and accepted by the user, persisted in LocalStorage.	Medium
FR-4.4	The system shall allow the user to clear their history and favourites from the settings screen.	Low

3.5 Feedback and Rating

ID	Requirement Description	Priority
FR-5.1	The system shall allow the user to give a simple like/dislike (thumbs up/down) reaction to a suggested activity.	High
FR-5.2	The system shall use recorded feedback to adjust the ranking score of similar activities in future recommendations (see FR-2.2).	Medium
FR-5.3	The system shall allow an optional short text comment with feedback (stored locally; not sent anywhere in the current phase).	Low

3.6 User Profile and Settings

ID	Requirement Description	Priority
FR-6.1	The system shall allow the user to set a display name (optional, local only) shown on the home screen.	Low
FR-6.2	The system shall allow the user to toggle a light/dark theme.	Medium
FR-6.3	The system shall allow the user to select categories they want to exclude from suggestions.	Medium
FR-6.4	The system shall persist all settings in LocalStorage and reload them automatically on the next visit.	High

3.7 Notifications / Reminders (Optional Module)

ID	Requirement Description	Priority
FR-7.1	The system may show an in-app reminder banner encouraging the user to check in with their mood after a period of inactivity in the tab.	Low
FR-7.2	The system may (in a future phase, with backend support) send browser push notifications at user-configured times.	Low

4. Use Case Model

The primary actor of the system is the **General/Student User**. A secondary actor, the **Administrator**, maintains the activity catalogue during development. The table below summarizes the key use cases; a corresponding use case diagram should be created separately using UML notation for the final presentation.

Use Case ID	Use Case Name	Actor	Description
UC-01	Check In Mood	User	User selects current mood, energy, and available time.
UC-02	Get Suggestion	User	System generates and displays a matching activity suggestion.
UC-03	View Alternative Suggestion	User	User requests another suggestion if the first is not suitable.
UC-04	Browse Catalogue	User	User explores the full list of activities with search/filter.
UC-05	Manage Favourites	User	User adds/removes activities from their favourites list.
UC-06	Give Feedback	User	User rates a suggested activity as helpful or not.
UC-07	View History	User	User views a log of previously suggested/completed activities.
UC-08	Update Settings	User	User changes theme, excluded categories, or display name.
UC-09	Maintain Activity Catalogue	Administrator	Admin adds, edits, or removes activities in the dataset during development.

4.1 Sample Use Case Description — "Get Suggestion" (UC-02)

Field	Detail
Preconditions	User has completed the mood check-in (UC-01).
Main Flow	1. System reads mood, time, and category inputs. 2. Recommendation engine filters and ranks the catalogue. 3. Top-ranked activity is displayed with a short description and duration. 4. User may accept, favourite, or request an alternative.
Alternate Flow	If no activity matches all filters, the system relaxes the least important filter (e.g., category) and retries before showing a generic fallback activity.
Postconditions	The suggested activity is logged in the session history.

5. External Interface Requirements

5.1 User Interfaces

- **Home / Mood Check-in Screen:** Central entry point with mood selector, time selector, and a prominent "Get Suggestion" call-to-action button.
- **Suggestion Screen:** Displays the recommended activity as a card with title, description, duration, icon, and action buttons (Accept, Another, Favourite, Feedback).
- **Catalogue Screen:** Grid/list of all activities with a search bar and filter chips.
- **Favourites Screen:** List of saved activities with quick access to re-suggest them.
- **History Screen:** Timeline/list view of past suggestions.
- **Settings Screen:** Theme toggle, excluded categories, data reset option.
- The interface shall follow a calm, minimal visual style (soft green palette, rounded cards, ample white space) appropriate to the wellbeing-oriented purpose of the app, and shall be designed mobile-first with breakpoints for tablet and desktop.

5.2 Hardware Interfaces

No special hardware interfaces are required. The system runs entirely within a standard web browser on any device (desktop, laptop, tablet, or smartphone) with a display, and does not require sensors, external peripherals, or dedicated hardware.

5.3 Software Interfaces

Component	Purpose
Web Browser	Runtime environment for the frontend application (Chrome, Firefox, Edge, Safari — latest two major versions).
Frontend Framework (React.js)	Component-based UI development and state management.
Browser LocalStorage API	Client-side persistence of preferences, favourites, and history.
Static Hosting Platform	Deployment target for the built application (e.g., Netlify, Vercel, GitHub Pages).
Future: REST API Backend	Planned interface for multi-device sync, exposing endpoints for activities, users, and feedback (see Section 11).

5.4 Communication Interfaces

In the current phase, the application requires only a standard HTTPS connection to load static assets (HTML, CSS, JS, and the activity dataset) from the hosting server; no further network communication takes place during use. When the future backend phase (Section 11) is implemented, the system will communicate with the server using HTTPS-based RESTful API calls exchanging JSON payloads.

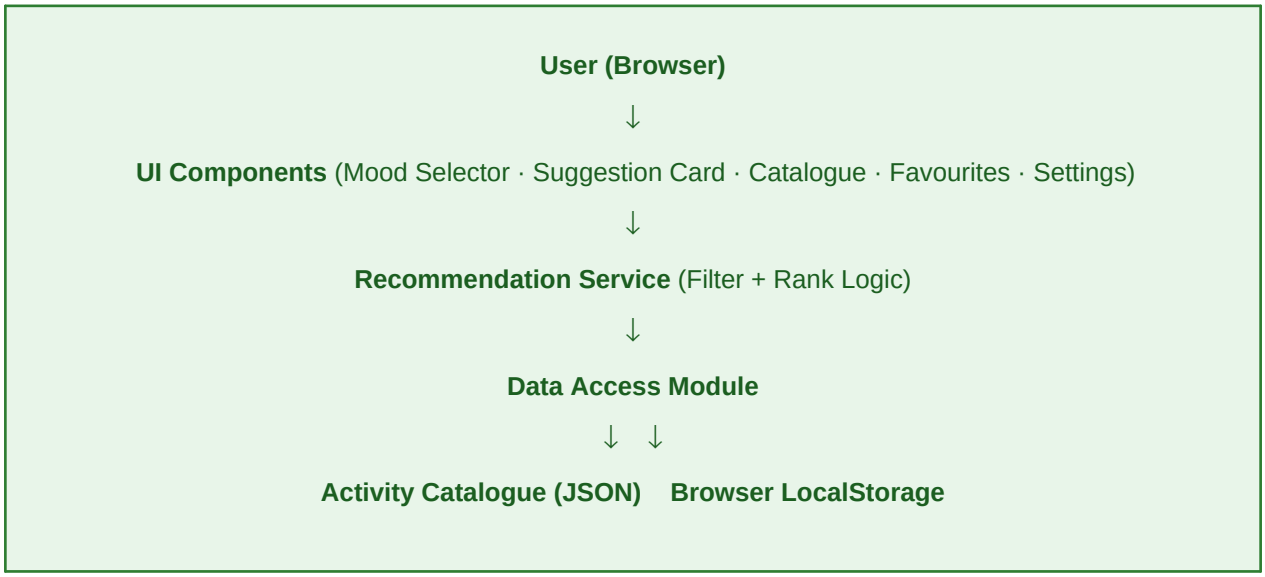
6. System Architecture

MoodLift follows a simple, layered client-side architecture for the current phase, designed to be cleanly extendable into a client-server architecture in the future phase without major rework of the UI layer.

6.1 Current Phase — Frontend-Only Architecture

Layer	Description
UI Components	Reusable React components: MoodSelector, SuggestionCard, CatalogueGrid, FavouritesList, HistoryList, SettingsPanel.
State Management	Local component state and React Context (or a lightweight store) holding current session data.
Recommendation Service	A pure JavaScript module implementing the filter-and-rank algorithm described in Section 3.2.
Data Access Module	Reads the static activity catalogue (JSON) and reads/writes user data to LocalStorage through a small wrapper (get/set/clear).
Activity Catalogue	Static JSON dataset bundled with the app, curated by the team (see Section 7.2 for structure).

6.2 Architecture Diagram (Conceptual)



6.3 Future Phase — Client-Server Architecture (Planned)

When the project is extended beyond the college submission, the Data Access Module can be swapped to call a REST API instead of reading local JSON, with minimal impact on UI components. The planned future stack is summarized below.

Component	Planned Technology
Backend Server	Node.js with Express.js exposing a REST API.
Database	MongoDB (document store) or PostgreSQL (relational) for users, activities, favourites, and feedback.

Component	Planned Technology
Authentication	Token-based authentication (JWT) for multi-device login.
Hosting	Cloud platform (e.g., Render, Railway, or similar) for the backend; static hosting continues for the frontend.

7. Data Requirements

7.1 Conceptual Data Model

Even though the current phase stores data in the browser rather than a formal database, the data is structured as if it were relational, so that migration to a real database in the future phase is straightforward. The core entities and their relationships are as follows:

- **User** (1) — has — (Many) **FavouriteActivity** records.
- **User** (1) — has — (Many) **HistoryEntry** records.
- **Activity** (1) — appears in — (Many) **HistoryEntry** and **FavouriteActivity** records.
- **Activity** (1) — receives — (Many) **FeedbackEntry** records.

7.2 Data Dictionary

Activity (static catalogue entity)

Field	Type	Description
activityId	String	Unique identifier for the activity.
title	String	Short display name, e.g., "5-Minute Breathing Exercise".
description	String	One to two sentence explanation of the activity.
category	Enum	Physical | Mental | Creative | Social | Relaxation.
moodTags	Array[String]	Moods this activity suits, e.g., ["Stressed", "Tired"].
durationMinutes	Number	Estimated time required in minutes.
icon	String	Reference to an icon/illustration asset.

UserPreference (LocalStorage entity)

Field	Type	Description
displayName	String	Optional local display name.
theme	Enum	Light | Dark.
excludedCategories	Array[String]	Categories the user does not want suggested.
favourites	Array[activityId]	List of favourited activity IDs.
history	Array[HistoryEntry]	Chronological list of past suggestions.

HistoryEntry / FeedbackEntry

Field	Type	Description
activityId	String	Reference to the suggested activity.
timestamp	DateTime	When the suggestion was made/accepted.
moodAtTime	String	The mood selected when this suggestion was generated.
feedback	Enum	Like | Dislike | None.

8. Non-Functional Requirements

8.1 Performance Requirements

ID	Requirement Description	Priority
NFR-1.1	The application shall load its initial screen within 3 seconds on a standard broadband connection.	High
NFR-1.2	Activity suggestions shall be generated and rendered within 1 second of user input, as all processing is client-side.	High
NFR-1.3	The application shall remain responsive (no visible lag) while filtering a catalogue of up to 200 activities.	Medium

8.2 Usability Requirements

ID	Requirement Description	Priority
NFR-2.1	A first-time user shall be able to receive their first activity suggestion within 3 taps/clicks from the home screen.	High
NFR-2.2	The UI shall follow a consistent, calming green-and-white visual theme with legible typography (minimum 14px body text).	Medium
NFR-2.3	The application shall be usable on screen widths from 360px (mobile) to 1920px (desktop) without horizontal scrolling.	High
NFR-2.4	The application shall meet basic accessibility guidelines: sufficient colour contrast, keyboard navigability, and alt text for icons.	Medium

8.3 Reliability and Availability

ID	Requirement Description	Priority
NFR-3.1	The application shall not lose previously saved favourites or history data on browser refresh.	High
NFR-3.2	The application shall gracefully handle a missing or corrupted LocalStorage entry by resetting to safe defaults instead of crashing.	Medium
NFR-3.3	As a static site, the application shall have availability equal to that of its hosting provider (typically 99%+ uptime).	Low

8.4 Security Requirements

ID	Requirement Description	Priority
NFR-4.1	As no personal accounts exist in the current phase, no sensitive personal data (passwords, payment data) shall be collected or stored.	High
NFR-4.2	All data stored in LocalStorage shall remain on the user's own device and shall not be transmitted to any server in the current phase.	High
NFR-4.3	In the future backend phase, all client-server communication shall use HTTPS, and passwords shall be stored using industry-standard hashing (e.g., bcrypt).	Medium

8.5 Maintainability and Portability

ID	Requirement Description	Priority
NFR-5.1	The codebase shall be organised into clearly separated components/modules (UI, logic, data) to simplify future maintenance.	Medium
NFR-5.2	The activity catalogue shall be stored in a single, well-structured JSON file so new activities can be added without code changes.	High
NFR-5.3	The application shall run on any modern operating system through the browser, without platform-specific installation.	Medium

9. Other Requirements

- **Legal/Compliance:** The application shall include a short disclaimer clarifying that MoodLift provides general wellbeing suggestions and is not a substitute for professional mental health advice.
- **Localization:** The current phase shall support English only; the content structure shall allow future translation without redesign.
- **Branding:** The green colour theme shall be used consistently across the UI, documentation, and presentation materials to reinforce a calm, growth-oriented brand identity.

10. Testing Considerations

The following testing approach is recommended to validate the requirements defined in this document during development and before faculty evaluation.

Test Type	Focus Area	Example Test Case
Unit Testing	Recommendation engine logic	Given mood="Stressed" and time="15 min", verify only matching activities are returned.
Component Testing	UI components	Verify SuggestionCard renders title, duration, and action buttons correctly.
Integration Testing	Data flow between modules	Verify a favourited activity persists after a page reload (LocalStorage round-trip).
Usability Testing	End-to-end user flow	Observe a new user completing mood check-in to suggestion in under 3 clicks.
Responsive/Cross-browser Testing	Layout consistency	Verify layout on Chrome, Firefox, and a mobile viewport (360px width).
Regression Testing	Stability after changes	Re-run core suite after adding new activities to the catalogue.

11. Future Scope

The following enhancements are intentionally deferred beyond the current academic submission to keep the project scope realistic, but are documented here to show a clear growth path:

- Backend API and database integration (Node.js/Express + MongoDB or PostgreSQL) for persistent, multi-device accounts.
- User authentication and profile management (sign-up, login, password recovery).
- A smarter recommendation engine using lightweight machine learning (e.g., collaborative filtering) trained on aggregated, anonymized feedback.
- Mood tracking analytics dashboard showing trends over time (weekly/monthly mood charts).
- Push notifications and reminder scheduling.
- Integration with third-party services (calendar apps, fitness trackers) for context-aware suggestions.
- A native mobile application built with a cross-platform framework (e.g., React Native).
- Social/community features such as sharing favourite activities with friends.

12. Appendix

12.1 Sample Activity Catalogue Entries

Title	Category	Mood Tags	Duration
5-Minute Breathing Exercise	Relaxation	Stressed, Anxious	5 min
Quick Walk Outside	Physical	Tired, Bored, Stressed	15 min
Journal Your Thoughts	Mental	Sad, Anxious, Stressed	10 min
Doodle or Sketch	Creative	Bored, Happy	15 min
Call a Friend	Social	Sad, Bored, Lonely	20 min
Quick Bodyweight Workout	Physical	Energetic, Stressed	20 min

12.2 Traceability Note

Each functional requirement (FR-x.y) in Section 3 maps to one or more use cases in Section 4 and is verifiable through the test types outlined in Section 10. It is recommended that the development team maintain a simple requirements traceability matrix (spreadsheet) mapping FR IDs to implemented components and test cases as the project progresses.

12.3 Glossary Reference

See Section 1.3 (Definitions, Acronyms and Abbreviations) for the complete glossary used throughout this document.

— End of Document —